

Clausify — Application Report

A risk-aware contract analyzer built on the two trained CUAD models BRAC University · Dept. of CSE · Updated 2026-07-02

Clausify is the demonstration app for the project. You give it a contract; it runs the two trained models live and returns a **risk-prioritized** breakdown of the clauses it finds — High / Medium / Low — following the risk taxonomy defined in the project poster.

1. What the app does

1. You **add a contract** — either paste the text or upload a `.txt` file.
2. **Model 1 (Presence Classifier)** scans the whole contract and detects which of the 41 clause categories are present, with a confidence score for each.
3. **Model 2 (Span Extractor)** pulls the exact clause text out of the contract for each detected category.
4. Every detected clause is tagged with a **risk level (High / Medium / Low)** and a short plain-English reason, then the results are grouped and sorted by risk — highest first.

The output mirrors the poster’s “Risk-Prioritized Report”: the riskiest clauses (e.g. Uncapped Liability, IP Ownership Assignment, Non-Compete) surface at the top so a non-lawyer sees the “watch out before signing” items immediately.

2. How it works (end to end)

2.1 Input

- A **toggle** lets the user choose **Paste text** or **Upload a file** — only the chosen input shows.
- The uploaded/pasted contract text is read as UTF-8.

2.2 Windowing (matches training)

- The contract is sliced into **2,000-character windows with a 1,500-char stride** (500 overlap), identical to how the models were trained.
- For very long contracts the number of windows is capped (30) to keep the demo responsive.

2.3 Model 1 — Presence detection

- For each of the 41 categories, the app runs the exact **CUAD question** for that category paired with every window through the (**question, window**) classifier.
- It **max-pools** the per-window probabilities: a category is reported **present** if any window’s confidence is $\geq$ the threshold (a slider, default 0.50). The best-scoring window is remembered for Step 2.4.

2.4 Model 2 — Span extraction

- For each detected category, the QA model runs over that category’s best window (in ~1,200-char focus chunks) and returns the highest-confidence text span — the actual clause.

2.5 Risk classification (from the poster)

- Each detected category is mapped to a risk level using the poster’s taxonomy — **8 High, 22 Medium, 11 Low** (41 total), scored from the **weaker party’s** point of view — plus a one-line reason.
- Results are grouped **High** → **Medium** → **Low** and, within a group, sorted by confidence.

Risk	Count	Examples
High	8	Non-Compete, Uncapped Liability, IP Ownership Assignment, Exclusivity, Most Favored Nation, Liquidated Damages, Irrevocable/Perpetual License, Minimum Commitment
Medium	22	Cap on Liability, Audit Rights, Renewal Term, Termination for Convenience, Change of Control, Anti-Assignment, ...
Low	11	Governing Law, Document Name, Parties, Effective Date, Expiration Date, License Grant, Insurance, ...

3. What the user sees

- **Risk summary bar** — a count of High · Medium · Low clauses detected, plus how many of the 41 categories were found.
- **Risk-grouped clause cards** — each card shows:
 - the clause **name** and a colored **risk badge**,
 - the plain-English **reason** it carries that risk,
 - a **confidence bar** (Model 1), color-matched to the risk level,
 - the **extracted clause text** (Model 2).
- An expandable **table of all 41 categories** with their risk level and presence score.

The interface is a dark, premium theme: an animated gradient background, glassmorphism panels, a shimmering title, and color-coded (red / amber / green) risk styling.

4. Technical details

Item	Value
Framework	Streamlit
Model 1	DistilBertForSequenceClassification (window-level presence)
Model 2	DistilBertForQuestionAnswering (span extraction)
Inference device	CPU (portable; avoids GPU/MPS memory limits)
Window / stride	2,000 / 1,500 chars
Presence rule	max-pool over windows, threshold slider (default 0.50)
Risk taxonomy	8 High / 22 Medium / 11 Low, weaker-party view (from poster)
Typical runtime	~5–15 s per contract on a laptop after models load

The app loads the **actual trained weights** produced by the training notebook from the sibling deliverable folder:

```
../clausify-research/notebooks/outputs/presence_mil/final/ → Model 1
../clausify-research/notebooks/outputs/span/final/ → Model 2
```

(Override with the CUAD_MODELS_DIR environment variable if the models move.)

5. How to run

```
cd "clausify-app"
pip install -r requirements.txt      # first time only
streamlit run app.py
```

The browser opens at <http://localhost:8501>. Choose **Paste text** or **Upload a file**, add a contract (two test files ship in the folder: `sample_contract.txt`, `test_contract.txt`), then click **Analyze contract**.

6. Files

```
clausify-app/
|--- app.py                - Streamlit UI (input toggle + risk-grouped results, premium theme)
|--- model_utils.py        - loads both models, windows the contract, runs inference, risk taxonomy
|--- cat_questions.json    - the 41 category → CUAD-question strings the models expect
|--- sample_contract.txt   - a real CUAD contract to test with
|--- test_contract.txt     - a synthetic contract covering many clause types
|--- requirements.txt
|--- README.md
|--- APP_REPORT.md         - this report
```

7. Honest notes

- The app runs **Model 1 alone** for presence (not the AND-ensemble), so it can **over-detect** slightly. Raising the threshold slider (e.g. to 0.85) shows only high-confidence clauses.
- Backbones were downsized for laptop training (DistilBERT, not DeBERTa-v3), so span boundaries are approximate. The point of the demo is that both models **generalize to contracts they never saw** and that the output is organized by the poster’s risk scheme.
- Risk levels are a **fixed, transparent taxonomy** (per category), not a learned per-instance score — matching the poster’s “transparent risk taxonomy” contribution.
- **The quoted clause text is the least reliable thing on a card.** An end-to-end evaluation of the full pipeline on the 102 held-out contracts found that only **21.7%** of gold clauses survive all three stages: the category is detected 78.0% of the time, but the window the presence model selects contains the actual clause only 68.3% of the time, and extraction inside a correct window succeeds 39.7% of the time (versus 82.7% when handed a known answer-bearing window). Mean token-F1 drops 0.763 → 0.386. **Read a clause card as “this contract appears to contain a clause of this type — look here”, not as “this is the clause”.** The badge and category name are considerably more trustworthy than the sentence beneath them.
- **The confidence bar is not a probability.** The presence score has an Expected Calibration Error of 0.201 and is overconfident everywhere: detections scoring above 0.9 (a nearly full bar) correspond to the clause actually being present about **73%** of the time, and scores in the 0.5-0.6 band to under **20%**. The threshold slider still trades recall for precision in the right direction, but the number on it should not be read as a chance of correctness.
- **Latency, measured** (CPU, Apple M4, models loaded): shortest test contract 2.1 s; 25th percentile 13.3 s; **median 27.1 s**; 75th percentile 32.4 s; longest 32.0 s. An earlier version of this report said “5-15 seconds”, which was an impression from short inputs rather than a measurement.

- **The 30-window cap silently truncates long contracts.** The app reads at most the first 45,500 characters, and **38.4% of CUAD contracts (196 of 510) are longer than that.** Clauses past the cap cannot be detected at all, and none of the reported accuracy figures reflect this.
- **Running the transformer alone is, unexpectedly, the better choice for High-risk clauses.** The AND ensemble misses 37.5% of High-risk clauses against the transformer’s 15.9%. The demo’s configuration was chosen for packaging reasons, but it is the one we would now choose deliberately.
- **Privacy.** All inference is local; the uploaded `.txt` is read into memory, never written to disk, never logged, and gone when the session ends. That guarantee comes from running locally – hosted on a shared server the same code would place the contract in memory the user does not control. There is no authentication, no size limit, no retention policy, and no redaction. Adding PDF support would introduce parser attack surface; calling any remote model API would send the contract to a third party.
- **The risk taxonomy has not been reviewed by anyone with legal training.** It was written by the three of us. It is auditable line by line — every category carries a written reason — but no legal professional has yet agreed with it, and the app should not be presented as if one had.
- The plain-English sentence attached to a clause is a **pre-written category template**, not a summary of the specific clause. The fine-tuned summarizer reproduces one of 41 templates verbatim on 100% of inputs; against clause-specific references it scores ROUGE-L 0.116, no better than printing the template with no model at all.